Manipal School of Information Sciences

Linux OS and Scripting

ASSIGNMENT 02

ASSIGNMENT 02

Compare Permissions on Regular Files and Directories

Student Name: Ananya H C
Reg. No. : 261100690015

Date: 21 August 2026

Contents

List of Figures **ii**

List of Tables **iii**

1 Problem Statement **1**
1.1 Aim . . . 1
1.2 Objective . . . 1

2 Experimental Setup **1**

3 Creation of Test Environment **1**

4 Permission Testing on Regular File **2**
4.1 Permission 400 (r–) . . . 2
4.2 Permission 200 (-w-) . . . 3
4.3 Permission 100 (–x) . . . 4
4.4 Permission 500 (r-x) . . . 4
4.5 Permission 600 (rw-) . . . 5
4.6 Permission 700 (rwx) . . . 6

5 Permission Testing on Directory **6**
5.1 Permission 400 (r–) . . . 6
5.2 Permission 200 (-w-) . . . 7
5.3 Permission 100 (–x) . . . 8
5.4 Permission 500 (r-x) . . . 8
5.5 Permission 600 (rw-) . . . 9
5.6 Permission 700 (rwx) . . . 10

6 Testing and Observations **11**

7 Comparison of File and Directory Permissions **12**

8 Questions and Answers **12**
8.1 What does read (r) permission allow you to do with a regular file? . . . 12
8.2 What does read (r) permission allow you to do with a directory? . . . 13
8.3 What does write (w) permission mean for a regular file? . . . 13
8.4 What does write (w) permission mean for a directory? . . . 13
8.5 What does execute (x) permission mean for a regular file? . . . 13
8.6 Why is execute (x) permission important for accessing a directory? . . . 13
8.7 Which permission combination allows the owner to fully access a regular file? . . . 13
8.8 Which permission combination allows the owner to fully access a directory? 13
8.9 Why do the same r, w, and x permissions behave differently for a regular file and a directory? . . . 13

9 Final Verification **14**

10 Conclusion **14**

List of Figures

1 Creation and verification of the test environment 2
2 Regular file with 400 (r–) permission 3
3 Regular file with 200 (-w-) permission 3
4 Regular file with 100 (–x) permission 4
5 Regular file with 500 (r-x) permission 5
6 Regular file with 600 (rw-) permission 5
7 Regular file with 700 (rwx) permission 6
8 Directory with 400 (r–) permission 7
9 Directory with 200 (-w-) permission 7
10 Directory with 100 (–x) permission 8
11 Directory with 500 (r-x) permission 9
12 Directory with 600 (rw-) permission 10
13 Directory with 700 (rwx) permission 11
14 Final verification of the test environment 14

List of Tables

1 Experimental setup 1
2 Observed behavior of permissions on files and directories 12

1 Problem Statement

Create a regular file named `sample.txt` and a directory named `sample_dir`. Create at least one file inside `sample_dir`.

Apply the permissions 400, 200, 100, 500, 600, and 700 using `chmod`. Perform the specified operations on both the regular file and the directory and record the observations.

The exercise must be performed as a normal user without using `sudo`.

1.1 Aim

To compare the behavior of read, write, and execute permissions on regular files and directories using Linux file permissions.

1.2 Objective

To apply different permission combinations using `chmod` and observe how read, write, and execute permissions affect regular files and directories.

2 Experimental Setup

The experiment was performed in Kali Linux using the terminal. A separate directory was created to ensure that the permission experiments were performed in a controlled environment.

Component	**Details**
Operating System	Kali Linux
Interface	Terminal
User Type	Normal User
Permission Command	`chmod`
Regular File	`sample.txt`
Directory	`sample_dir`
File inside Directory	`inside.txt`

Table 1: Experimental setup

3 Creation of Test Environment

A separate directory named `permission_lab` was created. Inside it, the required regular file and directory were created. A file named `inside.txt` was also created inside the directory.

```
mkdir -p ~/permission_lab
cd ~/permission_lab

echo "This is a sample file." > sample.txt

```

```
mkdir sample_dir

echo "This is a file inside sample_dir." > sample_dir/inside.txt

ls -l
ls -l sample_dir
whoami
```

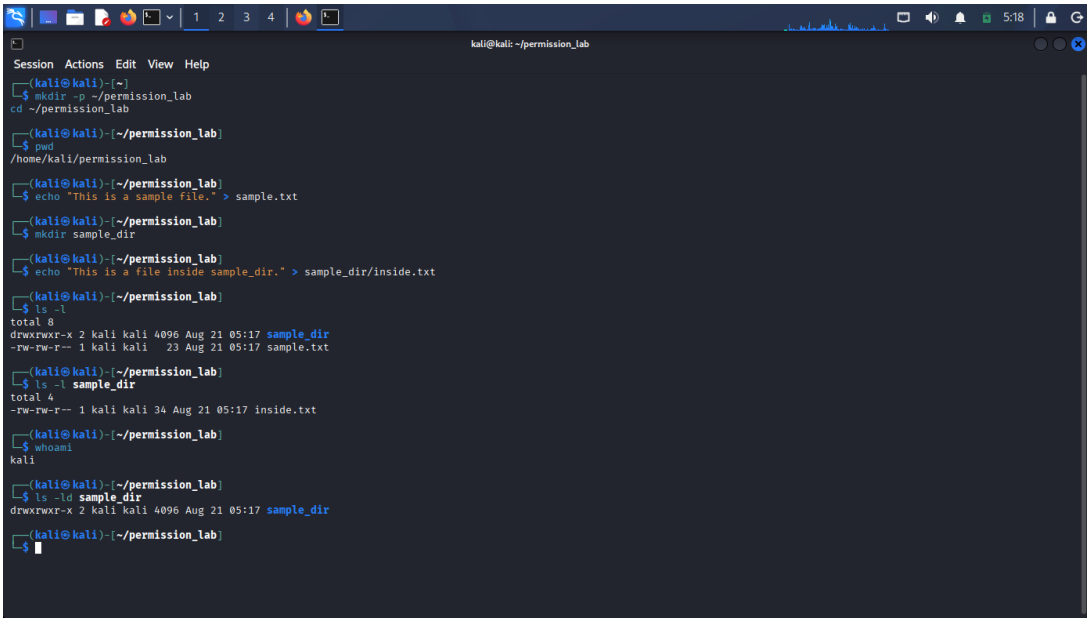


Figure 1: Creation and verification of the test environment

4 Permission Testing on Regular File

The file `sample.txt` was tested with the six permission combinations specified in the assignment.

4.1 Permission 400 (r–)

The permission was changed using:

```
chmod 400 sample.txt
ls -l sample.txt
cat sample.txt
echo "Modified text" > sample.txt
```

The file could be read successfully, while modification failed because write permission was not available.

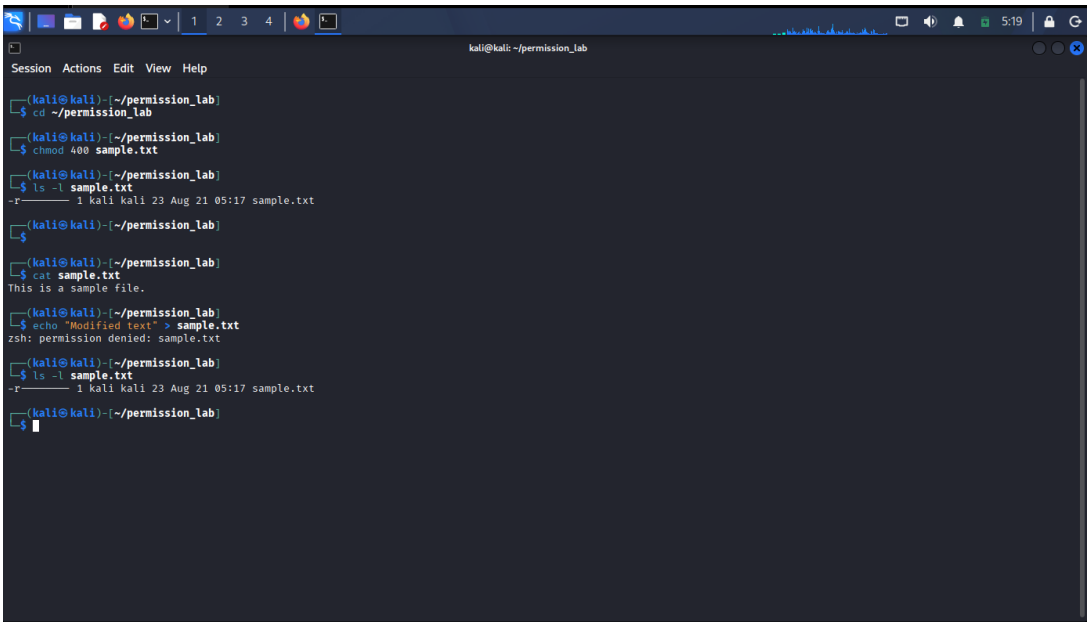


Figure 2: Regular file with 400 (r–) permission

4.2 Permission 200 (-w-)

The permission was changed using:

```
chmod 200 sample.txt
ls -l sample.txt
cat sample.txt
echo "Modified using write permission." > sample.txt
```

The file could not be read, but it could be modified because write permission was available to the owner.

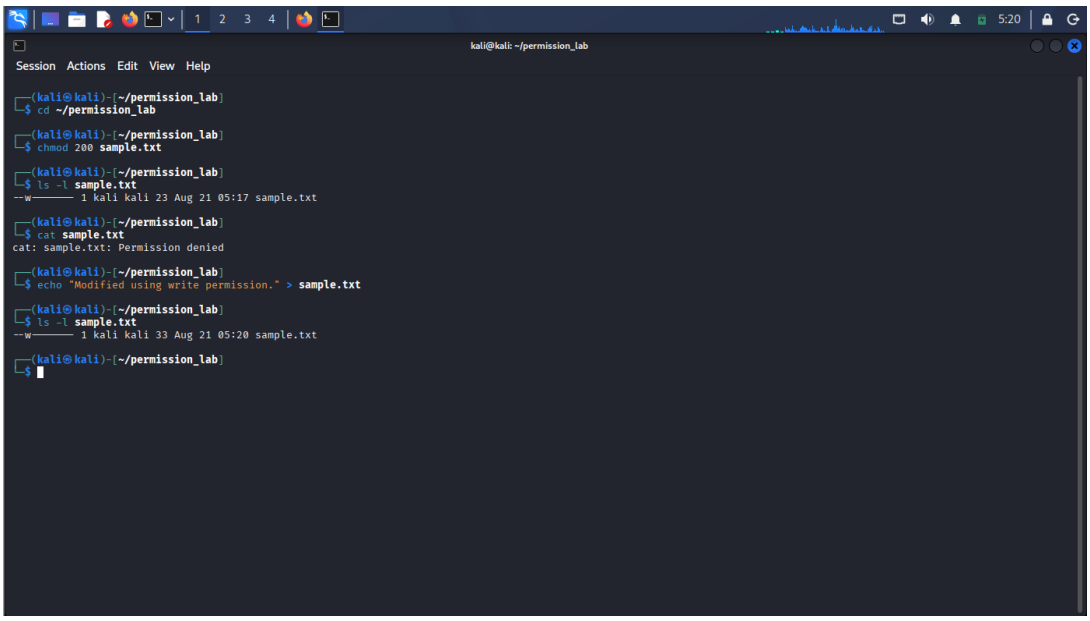


Figure 3: Regular file with 200 (-w-) permission

4.3 Permission 100 (–x)

The permission was changed using:

```
chmod 100 sample.txt
ls -l sample.txt
cat sample.txt
./sample.txt
```

The file could not be read. Execute permission was set, but the plain-text file was not a valid executable program.

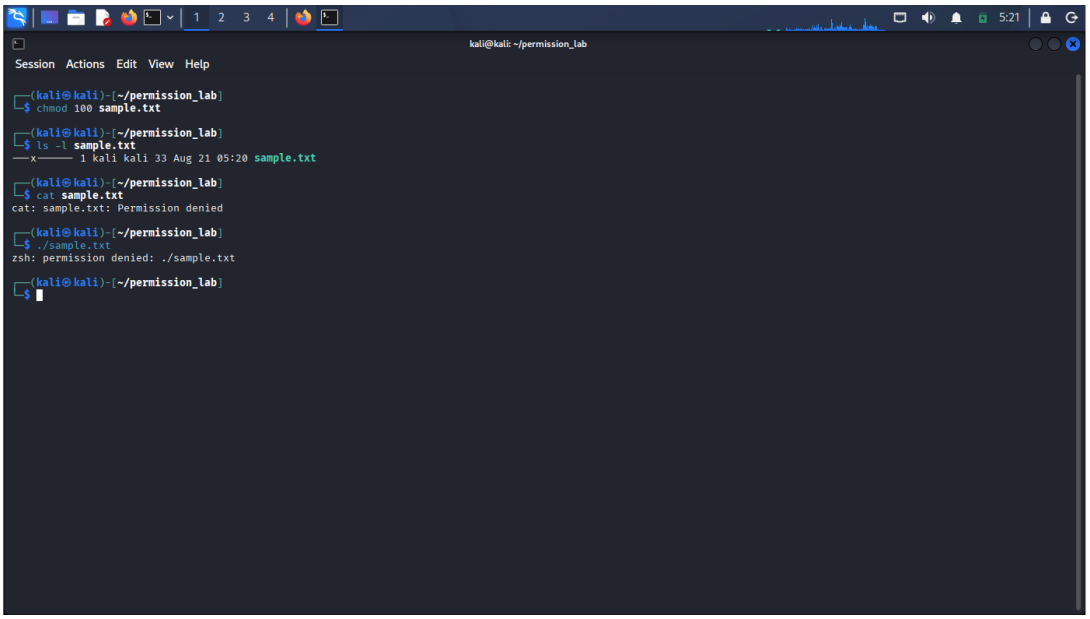


Figure 4: Regular file with 100 (–x) permission

4.4 Permission 500 (r-x)

The permission was changed using:

```
chmod 500 sample.txt
ls -l sample.txt
cat sample.txt
echo "Another modification" > sample.txt
./sample.txt
```

The file could be read, but modification failed because write permission was absent. Execute permission was present, although the file was not a valid executable program.

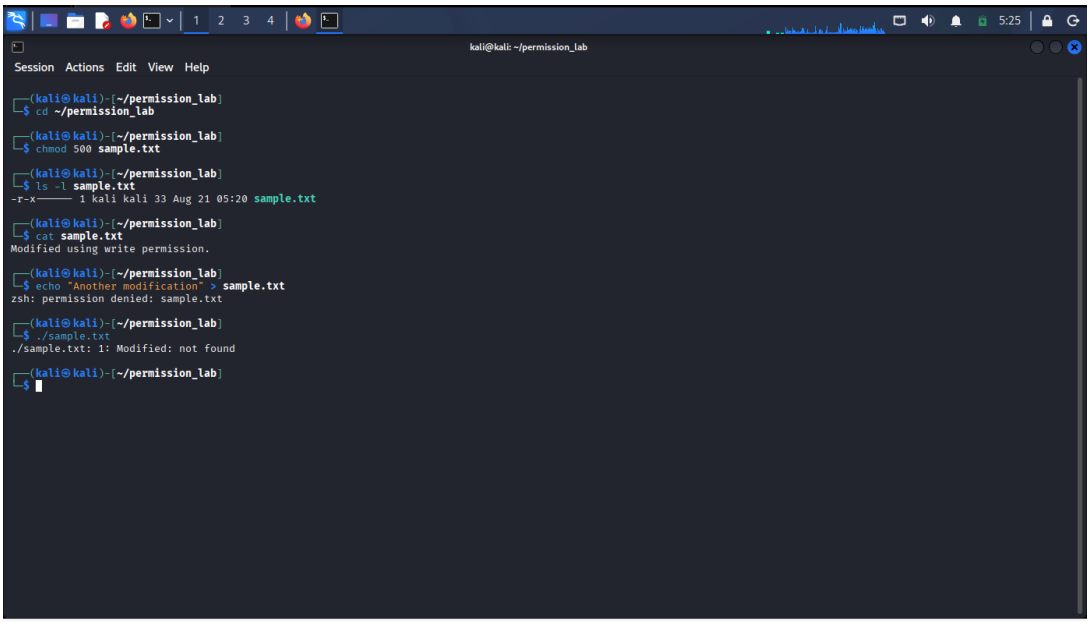


Figure 5: Regular file with 500 (r-x) permission

4.5 Permission 600 (rw-)

The permission was changed using:

```
chmod 600 sample.txt
ls -l sample.txt
cat sample.txt
echo "Modified with read-write permission." > sample.txt
cat sample.txt
```

The file could be read and modified because the owner had both read and write permissions.

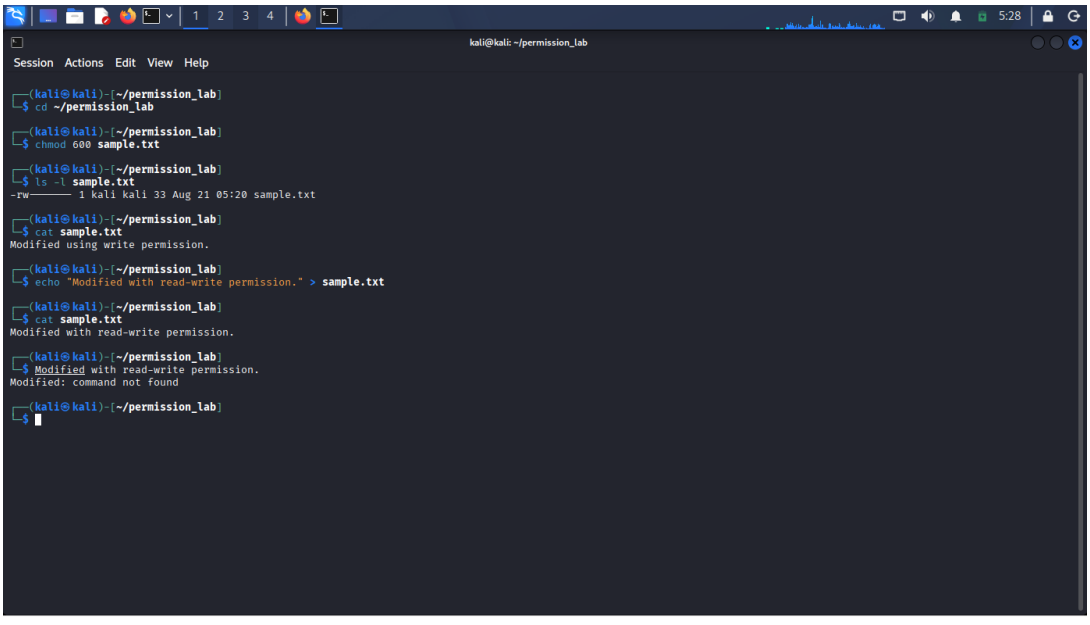


Figure 6: Regular file with 600 (rw-) permission

4.6 Permission 700 (rwx)

The permission was changed using:

```
chmod 700 sample.txt
ls -l sample.txt
cat sample.txt
echo "Modified with full permission." > sample.txt
cat sample.txt
./sample.txt
```

The owner had read, write, and execute permissions. The file could be read and modified. The execute permission was also present, although the current plain-text file was not a valid executable program.

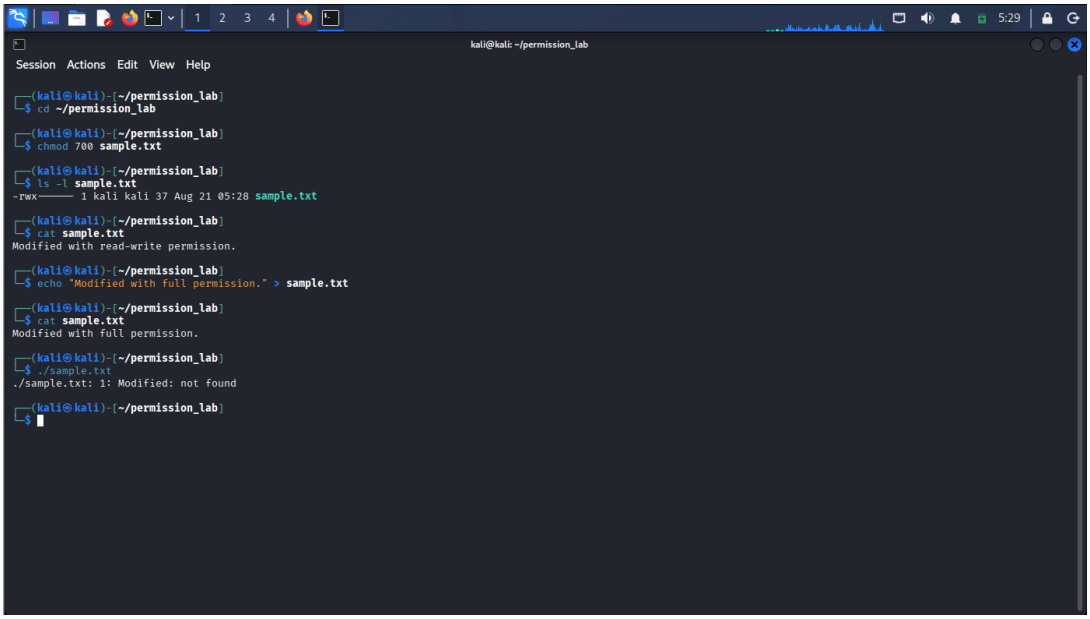


Figure 7: Regular file with 700 (rwx) permission

5 Permission Testing on Directory

The directory `sample_dir` was tested with the same six permission combinations. The tests focused on listing, entering, creating, and deleting files as specified in the assignment.

5.1 Permission 400 (r–)

The permission was changed using:

```
chmod 400 sample_dir
ls -ld sample_dir
ls sample_dir
cd sample_dir
```

The directory could not be listed or entered because execute permission was absent.

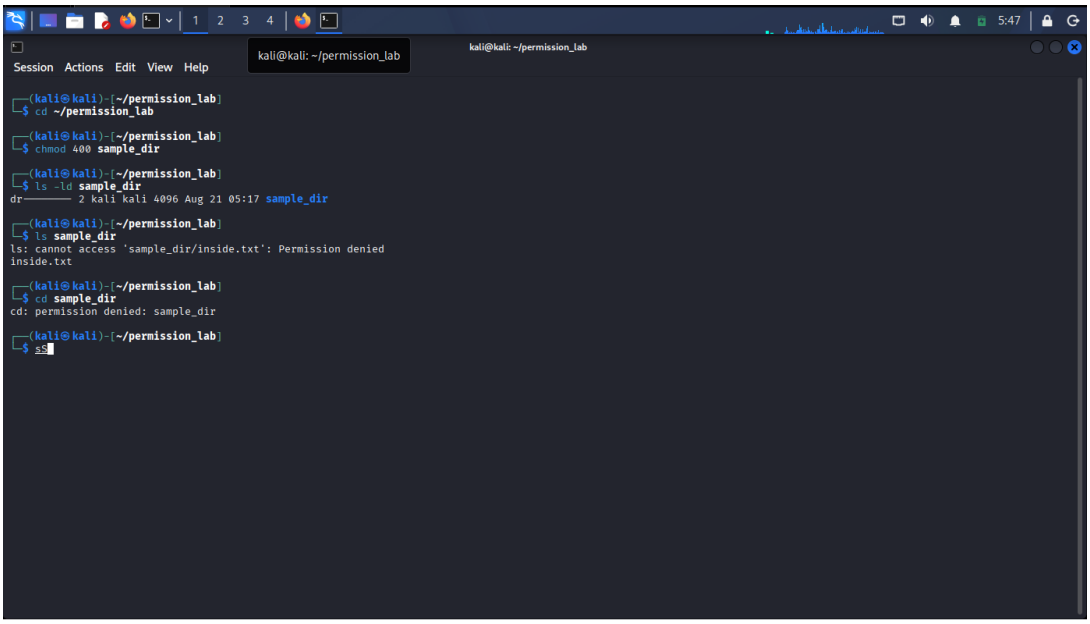


Figure 8: Directory with 400 (r–) permission

5.2 Permission 200 (-w-)

The permission was changed using:

```
chmod 200 sample_dir
ls -ld sample_dir
ls sample_dir
cd sample_dir
touch sample_dir/test200.txt
```

The directory could not be listed or entered. Write permission alone was not sufficient for normal directory access or creation of a file.

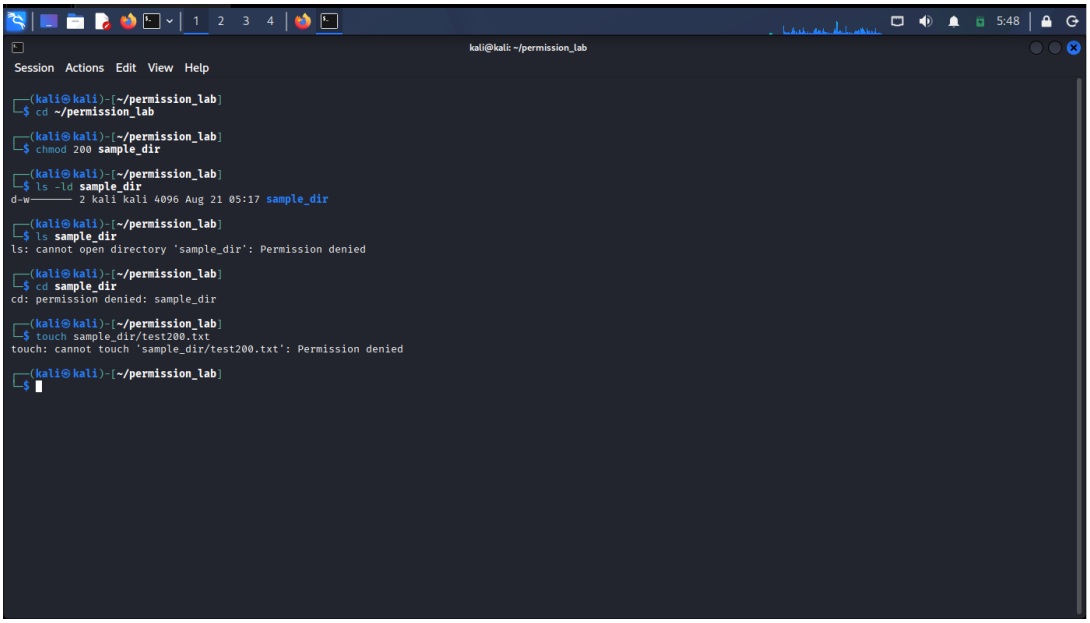


Figure 9: Directory with 200 (-w-) permission

5.3 Permission 100 (–x)

The permission was changed using:

```
chmod 100 sample_dir
ls -ld sample_dir
ls sample_dir
cd sample_dir
pwd
cd ..
cat sample_dir/inside.txt
```

The directory could not be listed because read permission was absent. However, it could be entered because execute permission was present.

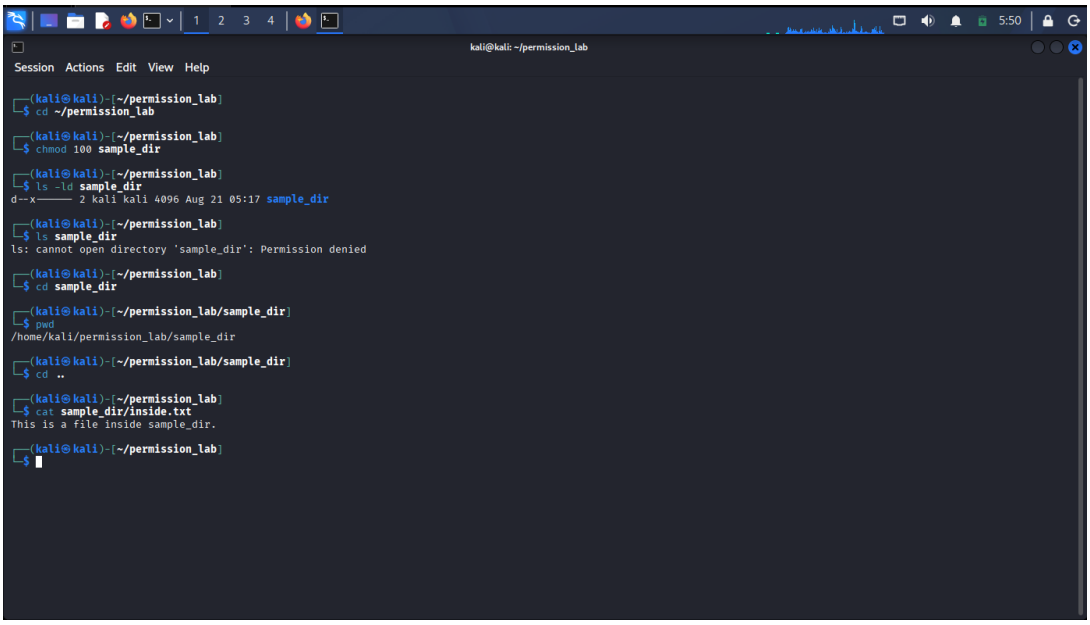


Figure 10: Directory with 100 (–x) permission

5.4 Permission 500 (r-x)

The permission was changed using:

```
chmod 500 sample_dir
ls -ld sample_dir
ls sample_dir
cd sample_dir
pwd
cd ..
touch sample_dir/test500.txt
```

The directory could be listed and entered because read and execute permissions were available. File creation failed because write permission was absent.

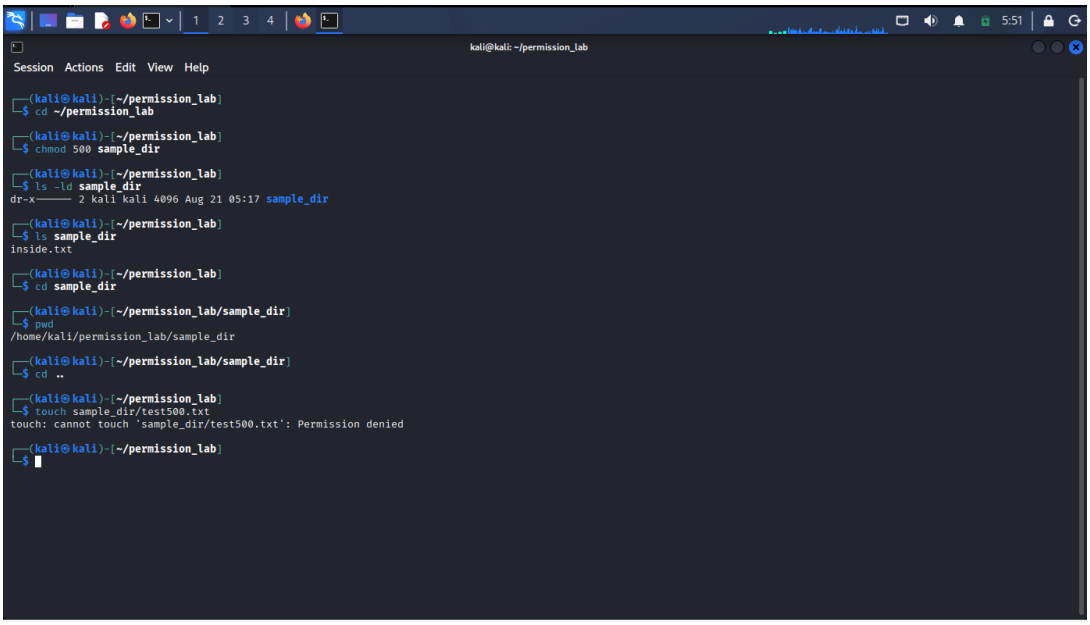


Figure 11: Directory with 500 (r-x) permission

5.5 Permission 600 (rw-)

The permission was changed using:

```
chmod 600 sample_dir
ls -ld sample_dir
ls sample_dir
cd sample_dir
touch sample_dir/test600.txt
```

Read permission was available, but without execute permission the directory could not be entered or its entries accessed normally. Write permission alone did not provide normal directory access.

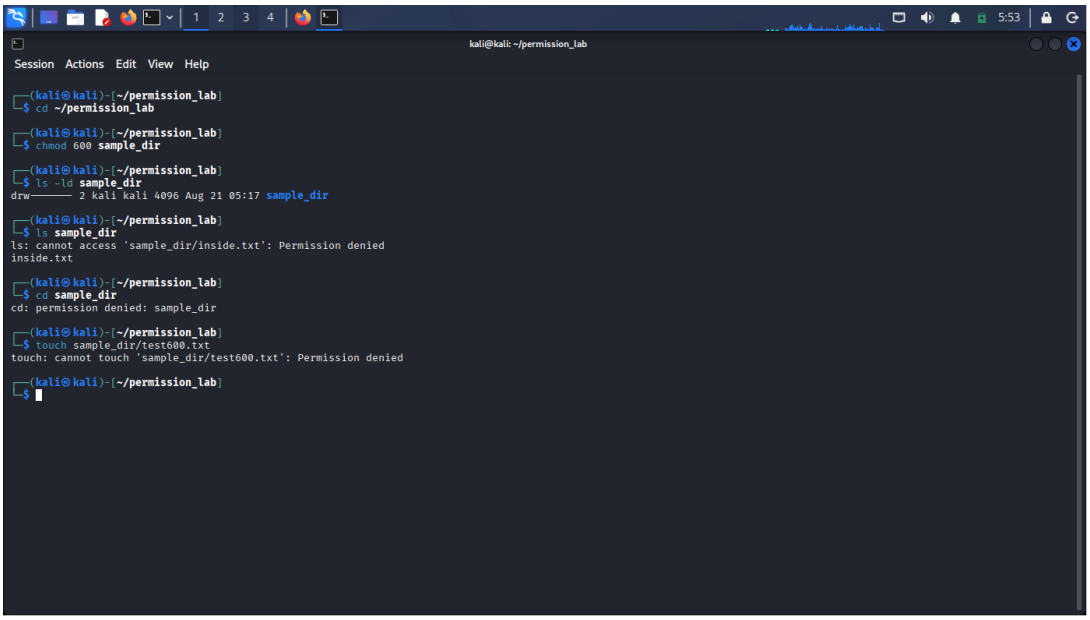


Figure 12: Directory with 600 (rw-) permission

5.6 Permission 700 (rwx)

The permission was changed using:

```
chmod 700 sample_dir
ls -ld sample_dir
ls sample_dir
cd sample_dir
pwd
cd ..
touch sample_dir/test700.txt
ls sample_dir
rm sample_dir/test700.txt
ls sample_dir
```

The directory could be listed and entered, and files could be created and deleted because the owner had read, write, and execute permissions.

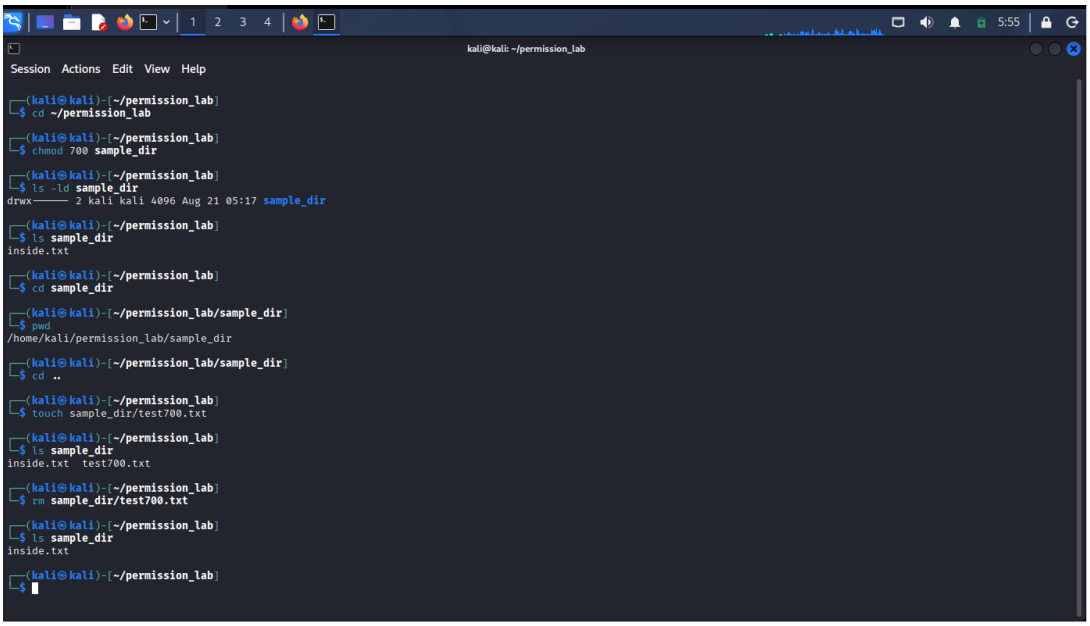


Figure 13: Directory with 700 (rwx) permission

6 Testing and Observations

The observations obtained from the permission experiments are summarized below.

Permission	Regular File (sample.txt)	Directory (sample_dir)
400 (r–)	Can read; cannot modify.	Cannot list or enter normally.
200 (-w-)	Cannot read; can modify.	Cannot list or enter; write alone is insufficient for normal access.
100 (–x)	Cannot read; execute bit is set, but the text file is not a valid executable.	Cannot list; can enter because execute permission is present.
500 (r-x)	Can read; cannot modify.	Can list and enter; cannot create files.
600 (rw-)	Can read and modify.	Read permission is present, but cannot enter normally because execute permission is absent.
700 (rwx)	Can read and modify; execute permission is also granted.	Can list, enter, create, and delete files.

Table 2: Observed behavior of permissions on files and directories

7 Comparison of File and Directory Permissions

The same permission symbols have different meanings depending on whether they are applied to a regular file or a directory.

For a regular file, read permission allows the contents of the file to be read, write permission allows the contents to be modified, and execute permission allows the file to be executed when it contains a valid executable program or script.

For a directory, read permission allows the directory entries to be listed, write permission permits modification of directory entries when the required access conditions are satisfied, and execute permission allows traversal and access to entries within the directory.

Therefore, directory permissions control access to the directory structure and its entries rather than the contents of a single file.

8 Questions and Answers

8.1 What does read (r) permission allow you to do with a regular file?

Read permission allows the contents of a regular file to be viewed or read.

8.2 What does read (r) permission allow you to do with a directory?

Read permission allows the names of entries within a directory to be listed, subject to the other required permissions.

8.3 What does write (w) permission mean for a regular file?

Write permission allows the contents of the regular file to be modified.

8.4 What does write (w) permission mean for a directory?

Write permission allows directory entries to be created, removed, or modified when the required directory access permissions are also available.

8.5 What does execute (x) permission mean for a regular file?

Execute permission allows a regular file to be executed as a program or script when its contents support execution.

8.6 Why is execute (x) permission important for accessing a directory?

Execute permission allows traversal of a directory and access to its entries. Without execute permission, a directory cannot be entered normally even when read permission is present.

8.7 Which permission combination allows the owner to fully access a regular file?

The permission combination **700 (rwx)** gives the owner read, write, and execute permissions on a regular file.

8.8 Which permission combination allows the owner to fully access a directory?

The permission combination **700 (rwx)** gives the owner read, write, and execute permissions on a directory, allowing listing, entering, creation, and deletion of entries.

8.9 Why do the same r, w, and x permissions behave differently for a regular file and a directory?

The permissions apply to different types of objects. For a regular file, the permissions control access to the file's contents and execution. For a directory, they control listing, modification of directory entries, and traversal. Therefore, the same permission symbols have different effects depending on the object to which they are applied.

9 Final Verification

After completing all permission tests, the permissions were restored to a clean final state using:

```
cd ~/permission_lab

chmod 600 sample.txt
chmod 700 sample_dir

ls -l
ls -ld sample_dir
ls -l sample_dir
```

The final state confirms that the test files and directory remain available with controlled owner permissions.

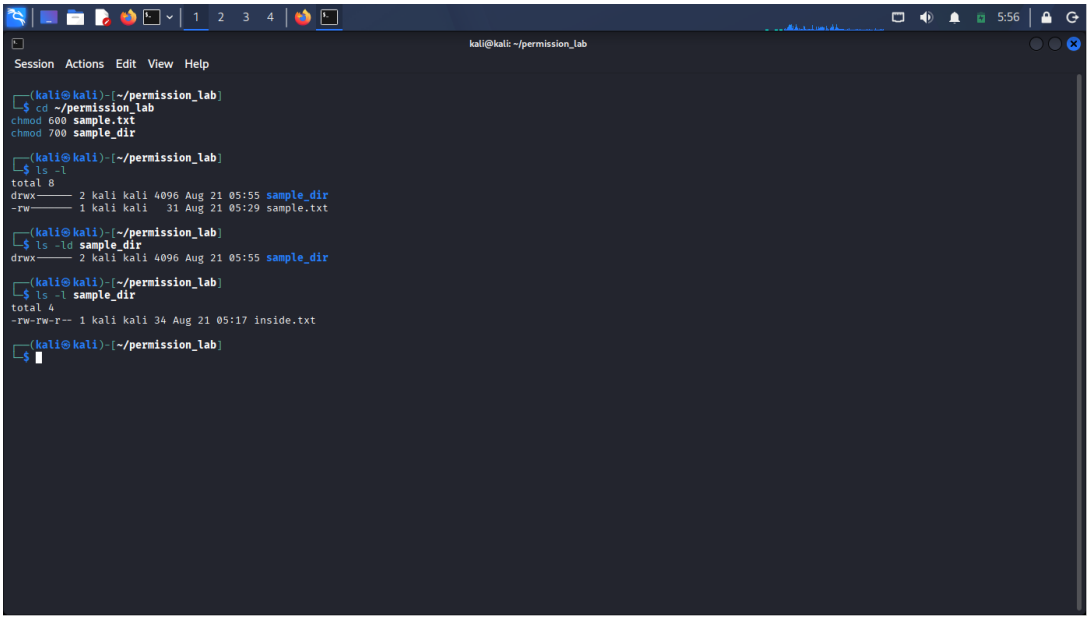


Figure 14: Final verification of the test environment

10 Conclusion

The experiment demonstrated that read, write, and execute permissions have different effects on regular files and directories. For files, the permissions control reading, modification, and execution, whereas for directories they control listing, modification of entries, and traversal. The experiments also showed that execute permission is essential for accessing a directory and that write permission alone is not sufficient for normal directory operations.

Thus, Linux file permissions provide controlled access to files and directories based on the assigned permission combination.